Single PDF RAG Assistant Interview Preparation

Getamesay Zegeye | 30 questions and project-specific answers

Use these answers to explain the implementation, debugging evidence and limitations. The reported 12/12 result applies only to a development/regression set on one PDF.

1 What does this project do

It answers questions about one uploaded PDF. The app extracts page text, builds local sentence embeddings, retrieves relevant chunks and asks Gemini to answer from those passages. The UI exposes page citations and retrieved text so the user can inspect support. It is a local educational application, not a deployed enterprise service.

2 What is RAG

Retrieval augmented generation combines information retrieval with a language model. Here, retrieval selects passages from the uploaded PDF and generation turns that context into an answer. RAG does not update model weights and does not guarantee correct answers; both retrieval coverage and faithful use of context must be evaluated.

3 Why did you choose Streamlit

It provides PDF upload, form inputs, status messages and evidence panels with little interface code. Session state retains the active document across reruns, and cache_resource avoids reloading MiniLM repeatedly. A shared production service would need additional authentication, concurrency controls, storage isolation and operational monitoring.

4 How is PDF text processed

pypdf extracts each page independently. Page metadata stays attached to every chunk, enabling citations. The implementation supports text PDFs; OCR is not included. Multi-column papers, formulas and tables can extract poorly, so retrieved text must be inspected rather than assuming extraction is faithful to the rendered page.

5 How does chunking work

The app uses fixed windows of 120 whitespace-separated words with 25-word overlap, advancing 95 words each time. Chunks do not cross page boundaries. These values are an explainable starting point, not an optimized universal setting. Overlap preserves some continuity but also creates duplicate content and extra embedding/storage work.

6 Why use overlap

A sentence or related fact can straddle a chunk boundary. Overlap makes it more likely that enough local context appears together. It cannot repair a page that retrieval never selects. In this project, same-page neighbor expansion helped continuity, but keyword retrieval was needed to recover the missing hardware page.

7 Which embedding model is used

The app uses sentence-transformers/all-MiniLM-L6-v2 on CPU to produce 384-dimensional vectors. The same model encodes document chunks and questions. It is compact enough for a small local app. A first-run model download is required, and token-heavy text can be truncated by the model despite word-based chunk limits.

8 Why move embeddings from Google to a local model

An earlier cloud-embedding workflow hit a daily free-tier quota. Local MiniLM removes embedding API quota usage and keeps document/query vectors on the computer. It still requires local compute and a model download. Gemini answer generation remains an external request with its own quota and data-transfer boundary.

9 What is stored in FAISS

FAISS stores normalized numeric vectors in IndexFlatIP. A separate JSON file stores chunk text and page numbers in matching order. Search returns vector IDs, which map back to those records. The index is exact, not approximate; the small single-document workload did not require an approximate index.

10 Why normalize embeddings

For unit-length vectors, inner product equals cosine similarity. Normalizing both chunk and question vectors makes IndexFlatIP ranking correspond to cosine similarity. A similarity score is not the probability that an answer is correct, so it should not be presented as model confidence.

11 How does persistence work

A SHA-256 key combines PDF bytes with the embedding model name and chunk settings. Each key has a FAISS file and JSON metadata under indexes/. Uploading the same PDF after restarting reloads its cache. Prompt or retrieval changes do not alter document vectors, while a different embedding model or chunking configuration needs a new index.

12 What is hybrid retrieval in this implementation

The system unions semantic FAISS candidates with keyword candidates. Keyword scores sum a log weight for each matching query term, with rarer terms receiving larger weights. This is a simple baseline, not BM25, learned ranking or reciprocal-rank fusion. Immediate same-page neighbors are added and duplicate chunk IDs removed.

13 How is k interpreted

k is the candidate count per retrieval method. With k=6, semantic search can contribute six and keyword search up to six more, before overlap and neighbors. The final passage count can exceed six. This implementation needs a token budget and final ranking stage before scaling to long PDFs.

14 Why sort selected chunks in document order

It makes adjacent evidence easier to read and preserves local context after combining candidate sources. The tradeoff is that high-relevance chunks are not necessarily first. For larger contexts I would compare relevance ordering, section grouping and token-budgeted passage packing using the same evaluation set.

15 What caused the Q7 failure

The exact prompt contained pages 5, 8 and 9 but omitted page 7, where the base-model hardware and training duration were stated. That is evidence of a retrieval coverage failure for that request. It would have been incorrect to conclude that the language model ignored evidence it had never received.

16 Why did prompt changes not solve Q7

Instructions cannot supply missing source evidence. More explicit prompts improved response focus but did not make page 7 appear. Debugging the actual generation context separated retrieval failure from generation failure. Hybrid candidates recovered the missing page, after which the shared answer correctly gave eight P100 GPUs, 100,000 steps and 12 hours.

17 How did you evaluate the project

The development set contains 12 questions about the Transformer PDF: seven direct, two synthesis and three missing-information questions. Answers were checked for correctness, completeness and citation support. Baseline shared outputs passed 11/12; after hybrid retrieval the developer reported 12/12. The final full transcript was not independently rerun in the packaging environment.

18 Does 12 of 12 imply 100 percent general accuracy

No. It is a small selected development/regression set used during tuning. It does not establish performance on unseen documents or questions. I would freeze the source and evaluation protocol, create held-out questions on a different PDF, and report failures, denominators and evidence provenance alongside any aggregate.

19 What is a false abstention

It is a refusal or missing-information response when the requested answer is available in the permitted document. Q7 initially produced one. By contrast, appropriate abstention occurs when the document does not provide the requested fact, such as an electricity cost or English-to-Amharic BLEU result in this test PDF.

20 How would you measure retrieval quality

For answerable questions, inspect whether returned chunks contain the required evidence. Track coverage of all needed facts or source pages, not just whether a citation appears in the answer. Precision, recall and ranking metrics require labeled relevance data. The project has not yet measured a formal retrieval hit rate or ranking benchmark.

21 How are hallucinations reduced

The prompt asks for answers supported by supplied passages, accurate page references and explicit missing details. Visible evidence supports manual review. These measures do not guarantee faithfulness. Further controls would validate cited pages, check claim support, add a calibrated relevance gate and test adversarial instructions embedded in PDFs.

22 How should citations be interpreted

The model supplies page numbers from context labels. They are not automatically verified. A correct statement can still have a wrong citation, and citing a few pages does not prove a fact is absent from the whole PDF. Evaluation should score factual content, citation support and completeness separately.

23 How are missing-information questions handled

The model is asked to say when the requested detail is absent from its context and answer any supported parts. There is no hard relevance threshold or deterministic abstention classifier. A robust next version would distinguish missing retrieved evidence from a verified absence across the document and evaluate false abstentions as well as hallucinations.

24 What happens during a 429 or 504 error

The app reports the API error instead of retrying indefinitely. A 429 indicates a quota or rate-limit issue; a 504 means the request did not finish before its deadline. The Google client uses a timeout and one attempt. Those settings do not guarantee success or an exact total wall-clock completion time.

25 Which data stays local and which leaves the computer

After the initial model download, document and query embeddings are computed locally. The FAISS cache and extracted text remain on disk. The question and retrieved passages go to Google for generation. API keys live in .env and are excluded from Git. This local setup does not provide multi-user security isolation.

26 Why not reuse the old Gemini embedding index

Embedding spaces from different models are incompatible even if vector dimensions happen to match. MiniLM must encode both the documents and the query for its index. The cache identity includes the embedding model and chunk settings to avoid silently reusing a differently configured index.

27 How would you support scanned or complex PDFs

Add OCR for scanned pages and layout-aware extraction for columns, tables and formulas. Preserve page provenance and evaluate extracted text against rendered pages. OCR and layout extraction introduce errors, compute cost and dependencies. They are future work rather than implemented features in this release candidate.

28 What code checks were performed

The packaging checks cover Python syntax and deterministic chunking, cache identity and retrieval behavior using controlled doubles. They do not download MiniLM or call Gemini. Local developer execution established that the app ran, but exact-source integration and browser acceptance checks must be repeated for this reconstructed release candidate.

29 How would you deploy this responsibly

First reproduce the exact release, then add authentication, per-user storage isolation, upload limits, request budgets and logging with appropriate data handling. Keep keys server-side. Test concurrency and dependency compatibility. The current portfolio site is a case study, not the hosted Python inference app, and no production service is claimed.

30 What would you improve next

I would evaluate new questions on an unseen PDF, record exact model revisions and dependency locks, and cap context tokens. Then compare BM25 or rank fusion and a reranker against the simple union baseline. Citation checking, OCR and multi-PDF retrieval would follow measured needs rather than being claimed as already implemented.